# Error Handling

A single response envelope, the exception → HTTP status map, and how the service layer raises errors.

---

Every error in the backend funnels through one `@RestControllerAdvice` (`com.finance.common.exception.GlobalExceptionHandler`) and comes back as a single, predictable `ErrorResponse`. Controllers never build error bodies; services throw a typed exception and the handler maps it to the right HTTP status, error code and message.

## Response shape

```
{
  "success": false,
  "message": "Portfolio 42 not found",
  "errorCode": "RESOURCE_NOT_FOUND",
  "timestamp": "2026-05-29T10:15:30",
  "path": "/api/v1/portfolios/42",
  "data": "null",
  "validationErrors": { "quantity": "must be greater than 0" }
}
```

| FIELD | MEANING |
|---|---|
| `success` | always `false` for an error |
| `message` | human-readable, localized through the `Translator` bean |
| `errorCode` | stable machine-readable code — branch on this, never on `message` |
| `timestamp` | server time the error was produced |
| `path` | the request URI that failed |
| `validationErrors` | present only on field-validation failures — `field → reason` |

## What raises which error

The interesting part is *why* an error happens. Each row below lists the condition that triggers the exception, so a `404` from a missing portfolio is never confused with a `404` from an unknown market symbol.

### 422 · `BusinessException` → `BUSINESS_ERROR`

A domain rule was violated even though the request was well-formed. Examples: selling a position that is already closed; reopening a position whose portfolio was deleted; an entry date in the future;

exceeding the per-portfolio lot limit; closing a derivative with a quantity larger than the open lot. These are *expected* business outcomes, not bugs — hence 422, not 400/500.

### 404 · `ResourceNotFoundException` → `RESOURCE_NOT_FOUND`

A persisted entity was looked up by id and not found **for this user** — e.g. `GET /portfolios/{id}` for an id you don't own, or a `positionId` that doesn't exist. Ownership misses return 404 (not 403) on purpose, so the API doesn't reveal that someone else's id exists.

### 404 · `SymbolNotFoundException` → `SYMBOL_NOT_FOUND`

A *market* lookup failed: an asset code / symbol that isn't in the tracked-asset registry, or has no price history. Distinct from `RESOURCE_NOT_FOUND` so clients can tell "your data is missing" from "that instrument doesn't exist".

### 400 · validation family

| EXCEPTION | RAISED WHEN | CODE |
|---|---|---|
| `MethodArgumentNotValidException` | a `@Valid` request body fails a constraint (blank `assetCode`, non-positive `quantity`, null `entryDate`) — the offending fields are returned in `validationErrors` | `VALIDATION_ERROR` |
| `ConstraintViolationException` | a `@Validated` path/query param fails (e.g. `page=-1`, `size` over the max) | `VALIDATION_ERROR` |
| `HttpMessageNotReadableException` | the JSON body is malformed / unparseable, or a number is sent where a date is expected | `MALFORMED_REQUEST` |
| `MissingServletRequestParameterException` | a required query parameter is absent | `MISSING_PARAMETER` |
| `BadRequestException` / `IllegalArgumentException` | input is syntactically valid but semantically wrong, caught in the service layer | `BAD_REQUEST` / `INVALID_ARGUMENT` |

### 409 · conflict family

| EXCEPTION | RAISED WHEN | CODE |
|---|---|---|
| `OptimisticLockingFailureException` | two writes hit the same row concurrently and the JPA `@Version` no longer matches — e.g. the same position edited from two tabs | `CONCURRENT_UPDATE` |
| `TaskAlreadyRunningException` | an admin data-refresh is triggered while the same single-flight task is still running | `TASK_ALREADY_RUNNING` |

| EXCEPTION | RAISED WHEN | CODE |
|---|---|---|
| `IllegalStateException` | an operation is attempted from an invalid state | `ILLEGAL_STATE` |

### 503 · `ExternalApiException` → `EXTERNAL_API_ERROR`

A downstream provider (EVDS, CoinGecko, Yahoo Finance, TEFAS, TCMB) failed, timed out, or returned an error envelope. Surfaced as 503 — the client should retry later rather than treat it as its own bad request.

### 500 · catch-alls

`RuntimeException` → `INTERNAL_ERROR` and `Exception` → `UNKNOWN_ERROR` are the safety net for anything unmapped. **These are the only handlers that log the full stack trace** (the com.finance logger routes it to OpenSearch via Kafka); everything above is logged at a lower level because it is an expected, client-facing condition.

## Raising errors

Throw a typed domain exception from the service — never assemble an error DTO by hand:

```
var portfolio = portfolioRepository.findByIdAndUserSub(id, userSub)
        .orElseThrow(() -> new ResourceNotFoundException("portfolio.notFound", id));

if (position.isClosed()) {
    throw new BusinessException("position.alreadyClosed");
}
```

Validation stays declarative on the request DTO (`@NotNull`, `@Positive`, …); the handler turns a failure into a `400 VALIDATION_ERROR` with the per-field map automatically.

## Tracing a failure

`errorCode` + `path` pinpoint *what* and *where*; the `timestamp` lines the response up with the backend log. Because `com.finance` logs ship to OpenSearch, a 5xx can be found by `errorCode` and `path` in Dashboards without reproducing it.

> The contract is intentionally **not** RFC 7807 (`application/problem+json`). `ErrorResponse` mirrors the success `ApiResponse` envelope so the frontend handles both with one code path.